Sprawozdanie: Część 2 — Eliminacja obiektów zasłoniętych

Piotr Szepietowski i Stanisław Dutkiewicz

22 kwietnia 2026

1 Wymagania

Zadanie polegało na rozszerzeniu projektu z Części 1 o obsługę **zamkniętych wielokątów** oraz ich renderowanie z **eliminacją obiektów zasłoniętych**. Wymagane elementy:

- scena 3D złożona z nieprzenikających się obiektów (zbiór uczciwy),
- użycie kamery wirtualnej (ruch, obrót, ogniskowa),
- sortowanie głębokościowe wielokątów (algorytm malarski),
- wypełnione wielokąty rastrowane do okna Raylib,
- bez użycia Z-bufora i bez ray tracingu.

2 Założenia Geometryczne

Zakładamy scenę **uczciwą**: obiekty nie pokrywają się i nie przecinają. Dopuszczamy co najwyżej wspólne krawędzie lub wspólne wierzchołki. Formalnie dla dwóch różnych ścian F_i, F_j:

$$\mathrm{int}(F_i) \cap \mathrm{int}(F_j) = \varnothing,$$

co upraszcza poprawność porządku malarskiego.

3 Architektura Rozwiązania

Program podzielono na następujące warstwy:

- definicja geometrii (wierzchołki i ściany czworokątne),
- transformacja świat $\to$ kamera,
- culling (back-face, near plane, off-screen),
- wyznaczanie klucza głębokości i sortowanie malarskie,
- rasteryzacja wielokątów wypełnionych i rysowanie krawędzi.

4 Model Matematyczny Kamery

Dla punktu świata $P_w = (x, y, z)$, pozycji kamery $C = (c_x, c_y, c_z)$, kąta *yaw* ψ i *pitch* ϕ:

$$P_c = R_{\text{pitch}}\, R_{\text{yaw}}\, (P_w - C).$$

Użyte macierze obrotu:

$$R_{\text{yaw}} = \begin{bmatrix} \cos\psi & 0 & -\sin\psi \\ 0 & 1 & 0 \\ \sin\psi & 0 & \cos\psi \end{bmatrix}, \qquad R_{\text{pitch}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}.$$

W implementacji stosowana jest wersja rozwinięta skalarna:

$$\begin{aligned} \Delta x &= x - c_x, \quad \Delta y = y - c_y, \quad \Delta z = z - c_z, \\ x_1 &= \Delta x \cos\psi - \Delta z \sin\psi, \\ z_1 &= \Delta x \sin\psi + \Delta z \cos\psi, \\ y_2 &= \Delta y \cos\phi - z_1 \sin\phi, \\ z_2 &= \Delta y \sin\phi + z_1 \cos\phi. \end{aligned}$$

4.1 Rzut Perspektywiczny

Dla punktu w przestrzeni kamery $P_c = (X_c, Y_c, Z_c)$ oraz ogniskowej f:

$$x_s = \frac{W}{2} + f\frac{X_c}{Z_c}, \qquad y_s = \frac{H}{2} - f\frac{Y_c}{Z_c},$$

przy warunku widoczności $Z_c > z_{\text{near}}$.

5 Eliminacja Obiektów Zasłoniętych

5.1 Back-face Culling

Dla ściany o wierzchołkach (A, B, C, D) normalna wyznaczana jest jako:

$$\mathbf{n} = (B - A) \times (C - A).$$

Niech $\mathbf{m} = \frac{1}{4}(A + B + C + D)$ będzie środkiem ściany w układzie kamery. Ściana jest pomijana, gdy:

$$\mathbf{n} \cdot \mathbf{m} \geq 0.$$

5.2 Near i Off-screen Culling

- **Near:** jeśli dowolny wierzchołek ma $Z_c \leq z_{\text{near}}$, ściana nie jest rysowana.
- **Off-screen:** jeśli prostokąt otaczający rzut ściany nie przecina obszaru okna, ściana jest pomijana.

5.3 Algorytm Malarski z Kryterium Przedziałowym

Dla każdej ściany po transformacji do układu kamery wyznaczamy przedział głębokości:

$$z_{far} = \max(z_A, z_B, z_C, z_D), \qquad z_{near} = \min(z_A, z_B, z_C, z_D).$$

Ściany sortowane są malejąco według pary (z_{far}, z_{near}), czyli od najdalszych do najbliższych. Kryterium to jest bardziej stabilne niż sortowanie wyłącznie po centroidzie: w przypadku gdy dwie ściany mają ten sam z_{far}, rozstrzygający jest z_{near}.

6 Rasteryzacja Wielokątów Wypełnionych

Każdy czworokąt triangulowany jest na dwa trójkąty:

$$(A, B, C, D) \;\Rightarrow\; \triangle(A, C, B) \cup \triangle(A, D, C).$$

Pole trójkąta w 2D:

$$\mathcal{A}(\triangle PQR) = \frac{1}{2}\left|\det\begin{bmatrix} Q_x - P_x & Q_y - P_y \\ R_x - P_x & R_y - P_y \end{bmatrix}\right|.$$

Kolejność wierzchołków jest odwrócona względem naiwnej projekcji, ponieważ rzutowanie odwraca oś Y (wzór $y_s = H/2 - f\,Y_c/Z_c$), zamieniając nawijanie CCW na CW w układzie ekranowym OpenGL. Podanie wierzchołków w odwróconej kolejności przywraca wymagane nawijanie CCW. Wypełnienie jest jednokolorowe (*flat fill*), a krawędzie są dorysowywane konturem.

7 Złożoność Obliczeniowa

Dla F ścian:

- transformacje i culling: $\mathcal{O}(F)$,
- sortowanie (insertion sort): $\mathcal{O}(F^2)$,
- rasteryzacja i rysowanie: $\mathcal{O}(F)$ (pomijając koszt pikselowy GPU).

8 Szczegóły Implementacyjne

- Język: Zig.
- Biblioteka renderująca: Raylib.
- Sterowanie kamerą: W/S/A/D, Q/E, strzałki, Z/X.
- Tryb debug (klawisz Tab): wizualizacja ścian odrzuconych (back-face, near, off-screen).

9 Przykłady i Wyniki

Program obsługuje wczytywanie sceny z pliku tekstowego. Format pliku jest taki sam jak w Części 1: każda linia zawiera współrzędne dwóch przeciwległych wierzchołków prostopadłościanu $(x_0\ y_0\ z_0\ x_1\ y_1\ z_1)$.

```
PS C:\Users\szepiet33\Documents\Workspace\Grafika\Part_2> zig build run -- --boxes ..\Part_1\rubick.txt
```

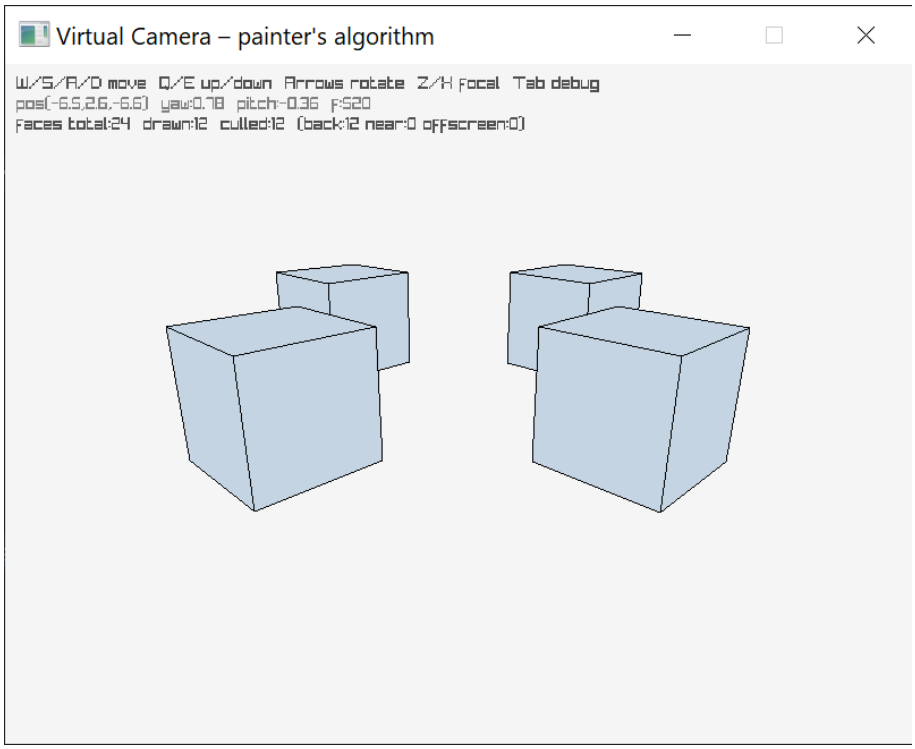
Rysunek 1: Przykład układu wczytanego z pliku.

Możliwe jest również uruchomienie programu bez podania pliku — wtedy wyświetlany jest domyślny układ czterech kostek.

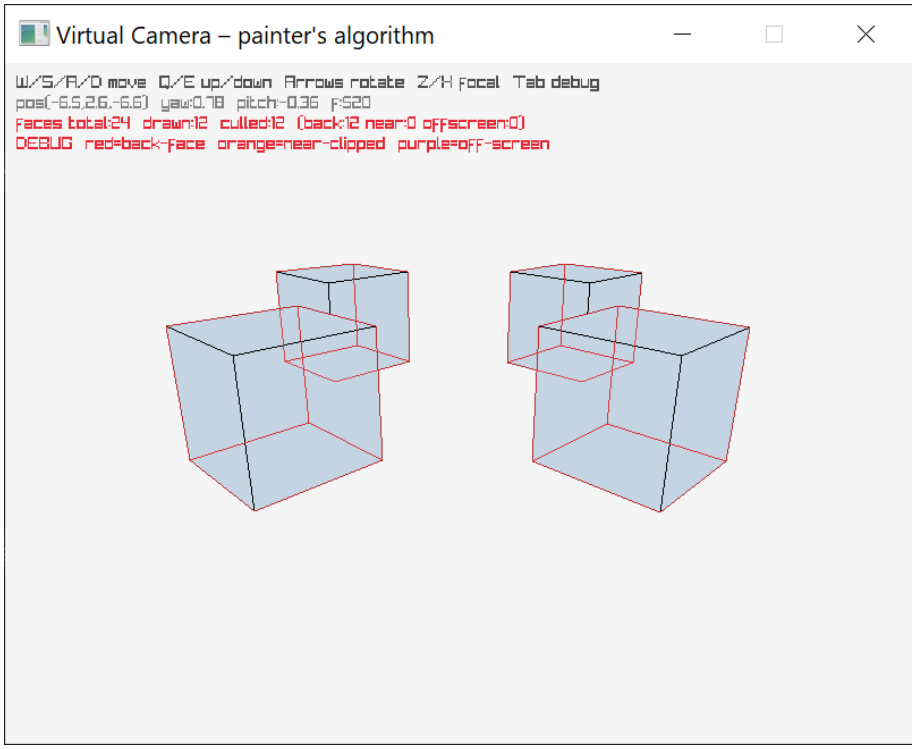
```
PS C:\Users\szepiet33\Documents\Workspace\Grafika\Part_2> zig build run
```

Rysunek 2: Przykład uruchomienia bez podania pliku z układem.

Przykładowe sceny bez podania pliku:

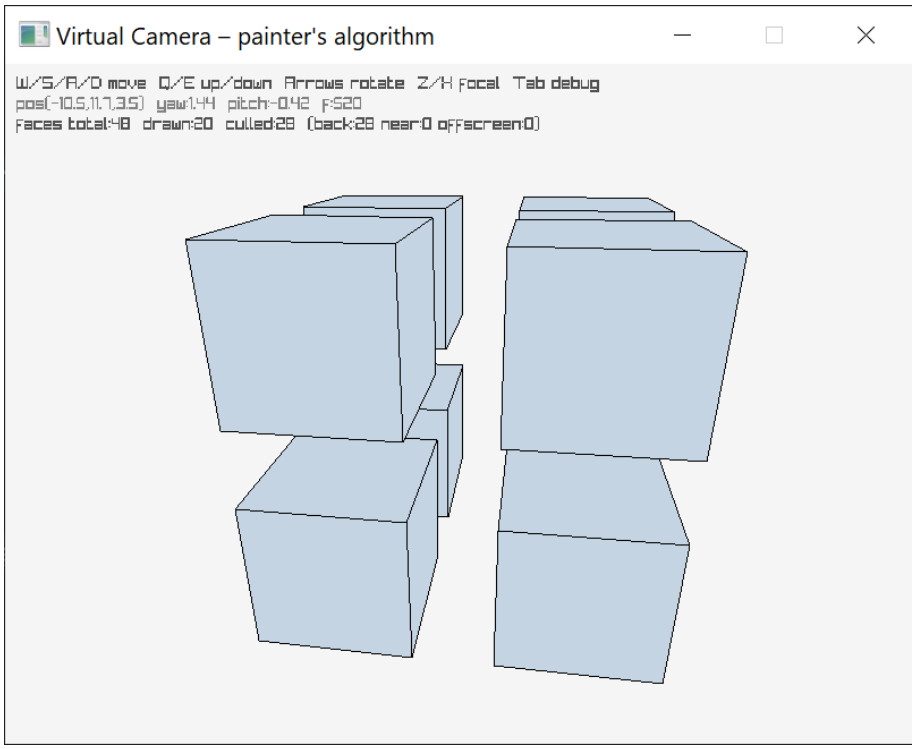


Rysunek 3: Przykładowy widok domyślnego układu czterech kostek.

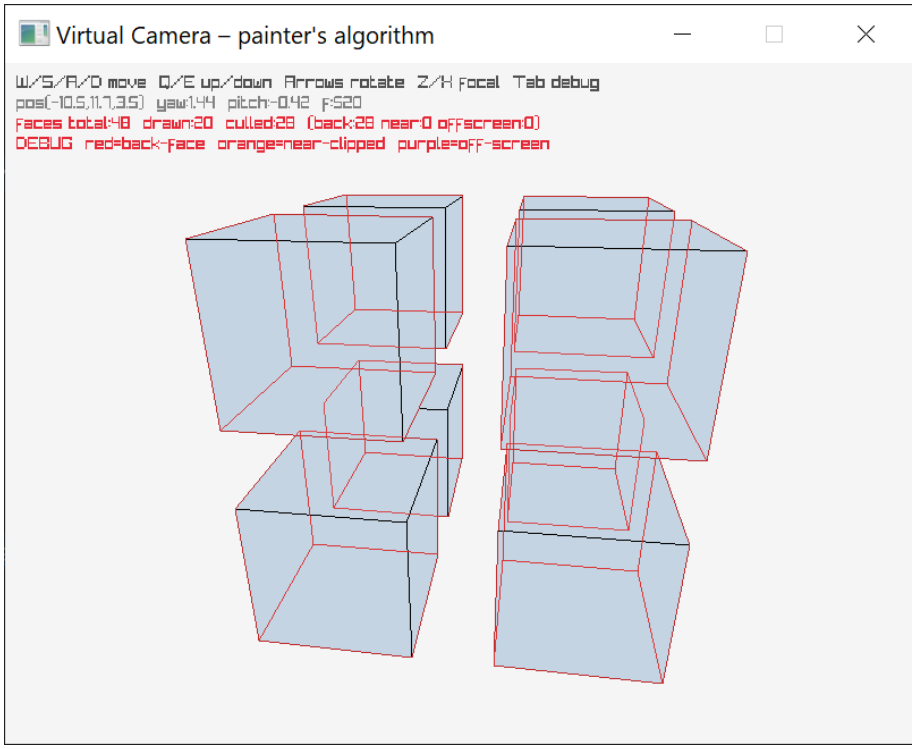


Rysunek 4: Ten sam widok w trybie debug.

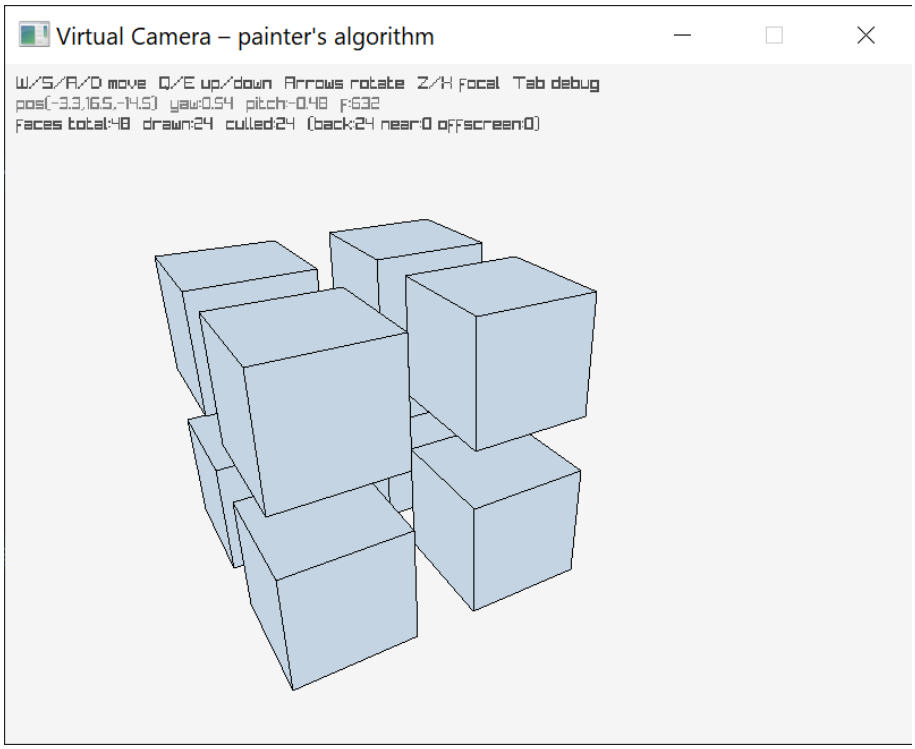
Przykładowe sceny z pliku `rubick.txt`:



Rysunek 5: Widok na kostkę Rubika powyżej przerwy między warstwami kostek.



Rysunek 6: Ta sama scena z uruchomionym trybem debug.



Rysunek 7: Widok na kostkę pod innym kątem z użyciem zoomu.

10 Potencjalne usprawnienia

- Zastąpienie insertion sort algorytmem o złożoności $\mathcal{O}(F \log F)$ dla dużych scen.
- Pełne klipowanie wielokątów na płaszczyźnie bliskiej zamiast odrzucania całej ściany.
- Obsługa wielokątów o dowolnej liczbie wierzchołków (nie tylko czworokątów).
- Wczytywanie formatów siatek 3D (np. OBJ, STL).
- Zastąpienie algorytmu malarskiego Z-buforem dla scen nieuczciwych (przecinające się obiekty).

11 Dyskusja Ograniczeń

Algorytm malarski działa poprawnie dla scen uczciwych. W przypadku wielokątów przecinających się wzajemnie może nie istnieć globalny porządek liniowy; wtedy wymagane są bardziej zaawansowane techniki (np. dzielenie wielokątów lub Z-bufor).

12 Wnioski

Projekt rozszerzył kamerę i rzutowanie z Części 1 o pełny potok eliminacji zasłonięć dla wielokątów wypełnionych. Uzyskano poprawne renderowanie sceny bez użycia Z-bufora i ray tracingu, zgodnie z wymaganiami zadania.

13 Repozytorium

`https://github.com/Hadidomena/Grafika`